AST.C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* Copy a string */
static char *copy_string(const char *s)
{
    if (!s)
        return NULL;

    size_t n = strlen(s) + 1;
    char *p = malloc(n);

    if (!p) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    memcpy(p, s, n);
    return p;
}

/* Create a node */
ASTNode *create_node(ASTType type, const char *value)
{
    ASTNode *n = calloc(1, sizeof(*n));
```

```
    if (!n) {
        perror("calloc");
        exit(EXIT_FAILURE);
    }

    n->type = type;
    n->value = copy_string(value);

    return n;
}

/* Add a child */
void add_child(ASTNode *parent, ASTNode *child)
{
    if (!parent || !child)
        return;

    if (!parent->children) {
        parent->children = child;
        return;
    }

    ASTNode *p = parent->children;

    while (p->next)
        p = p->next;

    p->next = child;
}
```

```
/* Node constructors */
ASTNode *make_function(const char *name)
{
   return create_node(AST_FUNCTION, name);
}

ASTNode *make_parameter(const char *name)
{
   return create_node(AST_PARAMETER, name);
}

ASTNode *make_call(const char *name)
{
   return create_node(AST_CALL, name);
}

ASTNode *make_return(ASTNode *expr)
{
   ASTNode *n = create_node(AST_RETURN, "return");
   add_child(n, expr);
   return n;
}

ASTNode *make_binary(const char *op,
                     ASTNode *left,
                     ASTNode *right)
{
   ASTNode *n = create_node(AST_BINARY, op);
```

```
    add_child(n, left);
    add_child(n, right);
    return n;
}

ASTNode *make_identifier(const char *name)
{
    return create_node(AST_IDENTIFIER, name);
}

ASTNode *make_number(const char *text)
{
    return create_node(AST_NUMBER, text);
}

/* Free the AST */
void free_ast(ASTNode *node)
{
    while (node) {
        ASTNode *next = node->next;

        free_ast(node->children);
        free(node->value);
        free(node);

        node = next;
    }
}
```

```
/* Print the AST */
void print_ast(const ASTNode *node, int depth)
{
    while (node) {
        for (int i = 0; i < depth; i++)
            printf("  ");

        switch (node->type) {
            case AST_FUNCTION:
                printf("Function Definition: %s\n", node->value);
                break;
            case AST_PARAMETER:
                printf("Parameter: %s\n", node->value);
                break;
            case AST_CALL:
                printf("Function Call: %s\n", node->value);
                break;
            case AST_RETURN:
                printf("Return Statement\n");
                break;
            case AST_BINARY:
                printf("Expression: %s\n", node->value);
                break;
            case AST_IDENTIFIER:
                printf("Identifier: %s\n", node->value);
                break;
            case AST_NUMBER:
                printf("Number: %s\n", node->value);
                break;
```

```
        }

        print_ast(node->children, depth + 1);
        node = node->next;
    }
}

/* Read one line */
static int read_line(const char *prompt, char *buf, size_t size)
{
    printf("%s", prompt);
    fflush(stdout);

    if (!fgets(buf, (int)size, stdin))
        return 0;

    size_t len = strlen(buf);

    if (len > 0 && buf[len - 1] == '\n')
        buf[len - 1] = '\0';

    return 1;
}

/* Remove surrounding whitespace */
static char *trim(char *s)
{
    while (isspace((unsigned char)*s))
        s++;
```

```
    char *end = s + strlen(s);

    while (end > s && isspace((unsigned char)end[-1]))
        *--end = '\0';

    return s;
}

/* Parse comma-separated parameters */
static void add_parameters(ASTNode *fn, char *list)
{
    char *p = trim(list);

    if (!*p)
        return;

    char *start = p;

    for (;;) {
        if (*p == ',' || *p == '\0') {
            char saved = *p;
            *p = '\0';

            char *name = trim(start);

            if (*name)
                add_child(fn, make_parameter(name));
```

```
            if (saved == '\0')
                break;

            start = p + 1;
        }

        p++;
    }
}

/* Expression parser */
typedef struct {
    const char *s;
    int error;
} Parser;

static void skip_spaces(Parser *p)
{
    while (isspace((unsigned char)*p->s))
        p->s++;
}

static ASTNode *parse_expr(Parser *p);

static ASTNode *parse_primary(Parser *p)
{
    skip_spaces(p);

    if (*p->s == '(') {
```

```
    p->s++;

    ASTNode *n = parse_expr(p);
    skip_spaces(p);

    if (*p->s != ')')
        p->error = 1;
    else
        p->s++;

    if (p->error) {
        free_ast(n);
        return NULL;
    }

    return n;
}

if (isdigit((unsigned char)*p->s)) {
    const char *start = p->s;

    while (isdigit((unsigned char)*p->s))
        p->s++;

    size_t len = (size_t)(p->s - start);
    char *text = malloc(len + 1);

    if (!text) {
        perror("malloc");
```

```
        exit(EXIT_FAILURE);
    }

    memcpy(text, start, len);
    text[len] = '\0';

    ASTNode *n = make_number(text);
    free(text);

    return n;
}

if (isalpha((unsigned char)*p->s) || *p->s == '_') {
    const char *start = p->s;

    while (isalnum((unsigned char)*p->s) || *p->s == '_')
        p->s++;

    size_t len = (size_t)(p->s - start);
    char *name = malloc(len + 1);

    if (!name) {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    memcpy(name, start, len);
    name[len] = '\0';
```

```
        ASTNode *n = make_identifier(name);
        free(name);

        return n;
    }

    p->error = 1;
    return NULL;
}

static ASTNode *parse_product(Parser *p)
{
    ASTNode *left = parse_primary(p);

    while (!p->error) {
        skip_spaces(p);
        char op = *p->s;

        if (op != '*' && op != '/')
            break;

        p->s++;

        ASTNode *right = parse_primary(p);

        if (p->error || !right) {
            free_ast(left);
            free_ast(right);
            return NULL;
```

```
        }

        char oper[2] = {op, '\0'};
        left = make_binary(oper, left, right);
    }

    return left;
}

static ASTNode *parse_expr(Parser *p)
{
    ASTNode *left = parse_product(p);

    while (!p->error) {
        skip_spaces(p);
        char op = *p->s;

        if (op != '+' && op != '-')
            break;

        p->s++;

        ASTNode *right = parse_product(p);

        if (p->error || !right) {
            free_ast(left);
            free_ast(right);
            return NULL;
        }
```

```
        char oper[2] = {op, '\0'};
        left = make_binary(oper, left, right);
    }

    return left;
}

static ASTNode *parse_expression(const char *text)
{
    Parser p = {text, 0};

    ASTNode *n = parse_expr(&p);
    skip_spaces(&p);

    if (p.error || *p.s != '\0') {
        free_ast(n);
        return NULL;
    }

    return n;
}

/* Parse function-call arguments */
static int add_call_arguments(ASTNode *call, char *text)
{
    char *s = trim(text);

    if (!*s)
```

```
    return 1;

char *start = s;
int depth = 0;

for (char *p = s;; p++) {
    if (*p == '('
        depth++;
    else if (*p == ')') {
        depth--;

        if (depth < 0)
            return 0;
    }

    if ((*p == ',' && depth == 0) || *p == '\0') {
        char saved = *p;
        *p = '\0';

        ASTNode *arg = parse_expression(trim(start));

        if (!arg)
            return 0;

        add_child(call, arg);

        if (saved == '\0')
            break;
```

```
            start = p + 1;
        }
    }

    return depth == 0;
}

/* Main program */
int main(void)
{
    char function_name[128];
    char parameters[1024];
    char expression[1024];
    char call_name[128];
    char arguments[1024];

    if (!read_line("Function name: ",
                   function_name, sizeof(function_name)) ||
        !read_line("Parameters (comma-separated): ",
                   parameters, sizeof(parameters)) ||
        !read_line("Return expression: ",
                   expression, sizeof(expression)) ||
        !read_line("Function to call: ",
                   call_name, sizeof(call_name)) ||
        !read_line("Call arguments (comma-separated): ",
                   arguments, sizeof(arguments))) {
        fprintf(stderr, "Input error\n");
        return EXIT_FAILURE;
    }
```

```
char *fn_name = trim(function_name);
char *cn_name = trim(call_name);

if (!*fn_name || !*cn_name) {
    fprintf(stderr, "Function names cannot be empty\n");
    return EXIT_FAILURE;
}

ASTNode *fn = make_function(fn_name);
add_parameters(fn, parameters);

char *return_text = trim(expression);
ASTNode *expr = NULL;

if (*return_text) {
    expr = parse_expression(return_text);

    if (!expr) {
        fprintf(stderr, "Invalid return expression\n");
        free_ast(fn);
        return EXIT_FAILURE;
    }
}

add_child(fn, make_return(expr));

ASTNode *call = make_call(cn_name);
```

```
    if (!add_call_arguments(call, arguments)) {
        fprintf(stderr, "Invalid function-call arguments\n");
        free_ast(fn);
        free_ast(call);
        return EXIT_FAILURE;
    }

    printf("\n=== FUNCTION AST ===\n");
    print_ast(fn, 0);

    printf("\n=== FUNCTION CALL AST ===\n");
    print_ast(call, 0);

    free_ast(fn);
    free_ast(call);

    return EXIT_SUCCESS;
}
```